

Análise e Projeto de Software Orientado a Objetos

Diagramas de Sequência de Sistema

Leonardo S. Victorio

Faculdade de Computação - UFMS

2026/1



- 1 Introdução
- 2 Elementos do Diagrama de Sequência
- 3 Representação de Casos de Uso
- 4 Ligação da Interface com o Domínio
- 5 Estratégias Statefull e Stateless
- 6 Exceções em Diagramas
- 7 Padrão DTO
- 8 Conclusão



Utilização dos Casos de Uso Expandidos

Na atividade de análise, o texto dos casos de uso expandidos possui duas utilizações principais:

1. Modelo Conceitual

Atua como fonte de informação para encontrar **conceitos** (entidades e regras) do domínio (estudado na próxima aula).

2. Comportamento do Sistema

Fonte primária para encontrar as **operações** e **consultas** de sistema. Estas darão origem aos métodos que fazem a interface do sistema com o mundo externo.



Operações vs Consultas de Sistema

Para garantir reusabilidade (Meyer, 1988) e coesão, separamos o comportamento em:

Operações de Sistema

- Ativadas a partir de um evento (ação do usuário).
- Indicam fluxo de informação do exterior para o interior.
- **Alteram** informações gerenciadas pelo sistema.
- Por definição, **não** retornam dados.

Consultas de Sistema

- Correspondem à verificação de informação armazenada.
- O retorno pode ser o dado puro ou processado (ex: média, total).
- **Não** inserem, removem ou alteram dados.

Funcionalidade Efetiva

Operações + Consultas de sistema = totalidade das funções do sistema.

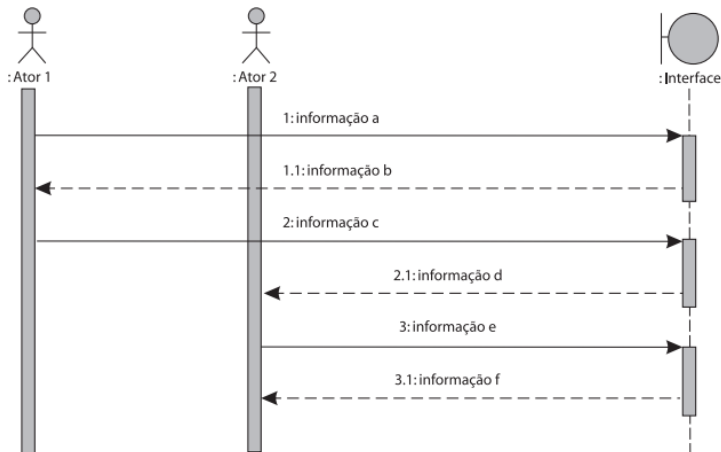
O Diagrama de Sequência de Sistema

A UML possui o **diagrama de sequência** para representar a linha do tempo dos eventos em um cenário de caso de uso.

- **Atores:** instâncias de usuários ou sistemas externos comunicantes. Sempre estão ativos.
- **Sistema:** representado como um único objeto do tipo caixa-preta (a Interface).
- **Linha de tempo:** linhas verticais (tracejada = inativo, cheia = ativo/aguardando).
- **Mensagens:** linhas horizontais representando o fluxo de informação.



Exemplo Visual do Diagrama



- Setas cheias: requisições/envio de informações para o sistema.
- Setas tracejadas: mensagens retornadas (o sistema apenas reage).



Mapeamento: Caso de Uso $\rightarrow$ Sequência

O diagrama de sequência sistematiza o caso de uso, refinando o funcionamento em duas etapas:

- 1 **Atores** $\leftrightarrow$ **Interface**: passos do caso de uso viram trocas de mensagem.
- 2 **Interface** $\leftrightarrow$ **Domínio**: chamadas para a controladora-fachada.

Passos do Caso de Uso

- **[IN]**: Envio de informação do ator para a interface.
- **[OUT]**: Envio de informação da interface para o ator (resposta).



Exemplo: Caso de Uso “Comprar livros”

Caso de Uso:

1. [IN] Informa identificação.
2. [OUT] Sistema informa livros disponíveis.
3. [IN] Seleciona livros.
4. [OUT] Sistema informa total e opções de endereço.
5. [IN] Seleciona endereço.
- ...

Caso de Uso: Comprar livros

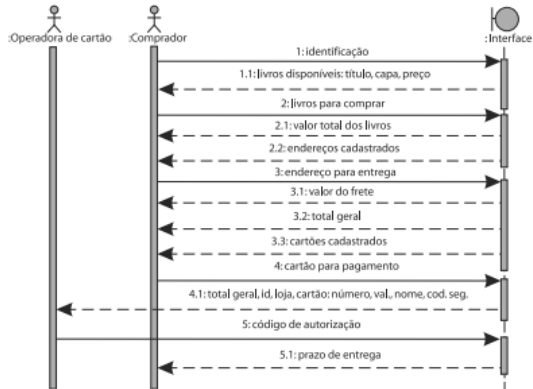
1. [IN] O comprador informa sua identificação. (1)
2. [OUT] O sistema informa os livros disponíveis para venda (título, capa e preço). (1.1)
3. [IN] O comprador seleciona os livros que deseja comprar. (2)
4. [OUT] O sistema informa o valor total dos livros e apresenta as opções de endereço cadastradas. (2.1 e 2.2)
5. [IN] O comprador seleciona um endereço para entrega. (3)
6. [OUT] O sistema informa o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento. (3.1, 3.2 e 3.3)
7. [IN] O comprador seleciona um cartão de crédito. (4)
8. [OUT] O sistema envia os dados do cartão e valor da venda para a operadora. (4.1)
9. [IN] A operadora informa o código de autorização. (5)
10. [OUT] O sistema informa ao comprador o prazo de entrega. (5.1)



Exemplo: Diagrama de Sequência para “Comprar livros”

Caso de Uso: Comprar livros

1. [IN] O comprador informa sua identificação. (1)
2. [OUT] O sistema informa os livros disponíveis para venda (título, capa e preço). (1.1)
3. [IN] O comprador seleciona os livros que deseja comprar. (2)
4. [OUT] O sistema informa o valor total dos livros e apresenta as opções de endereço cadastradas. (2.1 e 2.2)
5. [IN] O comprador seleciona um endereço para entrega. (3)
6. [OUT] O sistema informa o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento. (3.1, 3.2 e 3.3)
7. [IN] O comprador seleciona um cartão de crédito. (4)
8. [OUT] O sistema envia os dados do cartão e valor da venda para a operadora. (4.1)
9. [IN] A operadora informa o código de autorização. (5)
10. [OUT] O sistema informa ao comprador o prazo de entrega. (5.1)



Da Interface para a Camada de Domínio

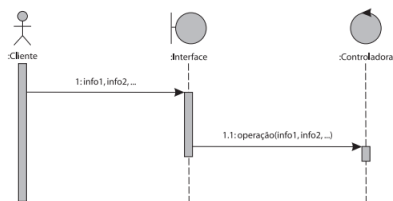
Ações do usuário na interface (ex: apertar botão, preencher formulário) geram chamadas a procedimentos computacionais na **camada de domínio**.

A camada de domínio é representada pela **Controladora-fachada**, que gerencia as operações e consultas acessíveis a partir da interface.

- **Evento de sistema** → Transforma-se em uma *operação* se altera dados, ou em *parâmetros* para uma consulta.
- **Resposta de sistema** → Requer a execução prévia de uma *consulta* de sistema.

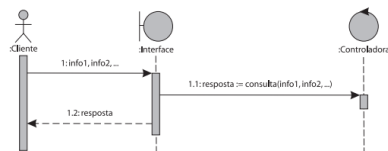


Operação de Sistema



Altera dados e não retorna valor
(exceto retornos para criar elementos
conceituais novos, ex: ID da compra).

Consulta de Sistema



Retorna informações solicitadas e
não altera dados armazenados.

Muitas vezes, uma transação (como uma compra) requer dados que o usuário envia no início (ex: o ID do comprador) ao longo de várias operações sucessivas.

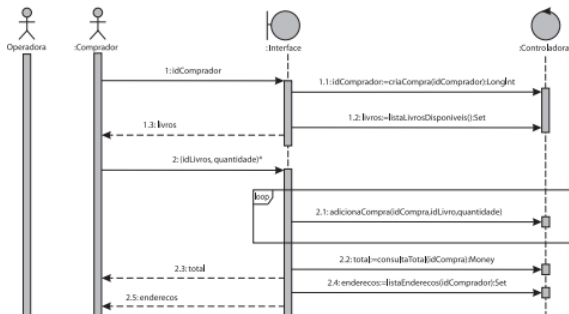
A Decisão de Projeto

Como a Controladora lida com esses parâmetros? Ela deve ter memória temporária (**Statefull**) ou receber todos os parâmetros novamente a cada chamada (**Stateless**)?



Estratégia Stateless (Sem Estado)

Nesta estratégia, cada vez que uma operação precisa da informação, a interface a repassa **explicitamente**.

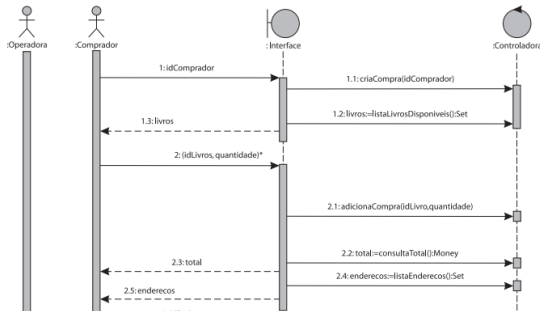


- **Vantagem:** controladora sem mecanismo de memória temporária.
- **Desvantagem:** grande passagem de parâmetros a cada chamada.



Estratégia Statefull (Com Estado)

A controladora implementa uma **memória temporária** para "lembrar" os dados contextuais (ex: compra corrente) e evitar o reenvio de parâmetros.



- **Vantagem:** informações transmitidas uma única vez pela rede.
- **Desvantagem:** exige memória temporária / associações transitórias.



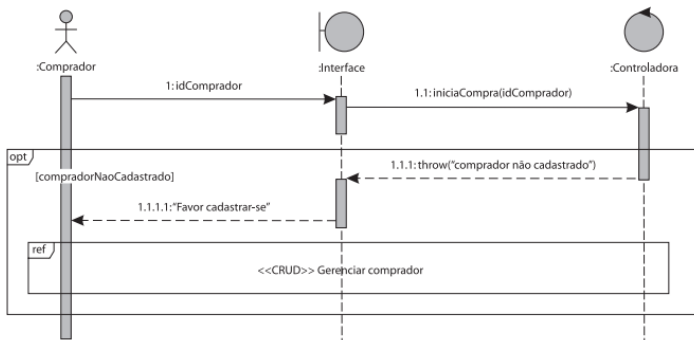
Passos alternativos do caso de uso normalmente representam **exceções** ou erros.

No Diagrama de Sequência

Uma exceção pode ser modelada como um **evento condicional** (usando fragmentos) que aborta a operação ou desvia o fluxo.

Exceções tipicamente ocorrem nas operações de sistema (consultas em geral retornam listas vazias ou resultados nulos em vez de abortar fluxos primários).

Representação de Fragmentos (opt e ref)

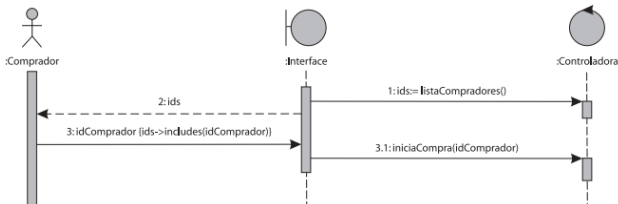


- **opt**: Bloco executado apenas se a condição for verdadeira.
- **ref**: Referência a outro diagrama, reduzindo complexidade e facilitando a leitura.

Alternativa: Exceção como Precondição

Outra abordagem é evitar que o erro atinja a operação, transformando a exceção em uma **precondição**.

- **Consulta Prévia:** Faz-se uma verificação *antes* da operação.
- **Seleção Fechada:** O ator seleciona dados de uma lista válida enviada pelo sistema, impossibilitando erros de digitação (ex: lista de cartões).



Data Transfer Object (DTO)

Rotular transições no diagrama com muitos atributos individuais (nome, endereço, CPF, telefone) é impraticável e poluído.

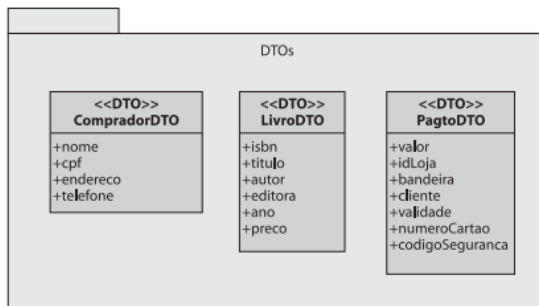
O que é um DTO?

Uma classe estilo registro/tupla contendo informações alfanuméricas. Diferente das classes conceituais, um DTO **não tem funcionalidade complexa**, possuindo apenas métodos de acesso (*getters* e *setters*).

- São encapsulamentos coesos de atributos, transferindo dados entre a interface e o sistema.
- Atuam como visões (*views* em banco de dados), isolando a camada conceitual de mudanças na interface.



Exemplo de Representação do DTO



Recomenda-se o uso do sufixo **DTO** (ex: **CompradorDTO**) para distinguir claramente das classes da camada de domínio.

Considerações Finais

- O diagrama de sequência de sistema foca no fluxo de dados entre os atores e a camada de aplicação.
- A sistematização revela informações e passos esquecidos no caso de uso.
- A identificação das ****operações**** e ****consultas de sistema**** prepara o terreno para a elaboração dos contratos (modelagem funcional).
- Decisões cruciais, como gerenciamento de estado (*statefull/stateless*) e empacotamento de dados (DTOs), são definidas neste momento.





WAZLAWICK, Raul Sidnei. *Análise e Projeto de Sistemas de Informação Orientados a Objetos*. 3. ed. Rio de Janeiro: Elsevier, 2015. Cap. 8 — Contratos.



LARMAN, Craig. *Utilizando UML e Padrões*. 3. ed. Porto Alegre: Bookman, 2007. Cap. 11 — Applying GoF Design Patterns.

Obrigado!

Dúvidas e Discussões

Leonardo S. Victorio

`leonardo.victorio@ufms.br`

Faculdade de Computação — UFMS

